

Draft Technical Report V1

Yacouba Diarra

Created on February 6 | Last edited on February 7

▾ Introduction

This report presents the findings from our Automatic Speech Recognition (ASR) training experiments on three different models with varying parameter sizes: **QuartzNet (19M parameters)**, **Parakeet Hybrid 114M TDT-CTC (114M parameters)**, and **Parakeet TDT-1.1B (1B parameters)**. The primary objectives of these experiments were:

1. Achieve a new state-of-the-art (SOTA) for open Bambara ASR models while ensuring full reproducibility.
2. Reduce model size compared to current SOTA models, which are mostly fine tuned versions of OpenAI's Whisper Large version models, to enable ASR deployment on edge devices without reliance on API calls.
3. Evaluate whether smaller models can outperform larger models when trained on a limited dataset (~35 hours of Bambara speech data).

We fine-tuned all models using **NVIDIA's NeMo toolkit** with the dataset RobotsMali/bam-asr-all, which consists of **~37 hours** of transcribed Bambara audio. The dataset is primarily derived (87%) from **RobotsMali's Jeli-ASR** dataset (Diarra et al, 2022).

▾ Dataset and experimental setup

The dataset used for fine-tuning contains transcribed Bambara speech audio with 87% of data sourced from the Jeli-ASR dataset. The training framework utilized NVIDIA's NeMo toolkit and all models were trained using bf16-mixed precision to optimize computational efficiency.

For all experiments, the models were trained with variations in learning rate schedules, batch sizes, data augmentation setup, optimizer settings, and decoder configurations. The Parakeet models utilize **Fast Conformer** architectures as encoders with **TDT (Token-and-Duration Transducer)** decoders (+ one auxiliary decoder for the hybrid model), while QuartzNet is a CNN-based model with **time-channel separable convolutions**.

▾ Model-Specific Experiments and Results

▾ Parakeet Hybrid 114M TDT-CTC

▼ Model Overview

Parakeet Hybrid 114M TDT-CTC is a hybrid ASR model leveraging the Fast Conformer architecture for its encoder and two independent decoders:

- **CTC (Connectionist Temporal Classification)** decoder, acting as an auxiliary decoder (initial weight = 0.3, this affects its contribution to the combined loss function which is used to train the whole network).
- **TDT (Token-and-Duration Transducer)** decoder, serving as the primary decoder.

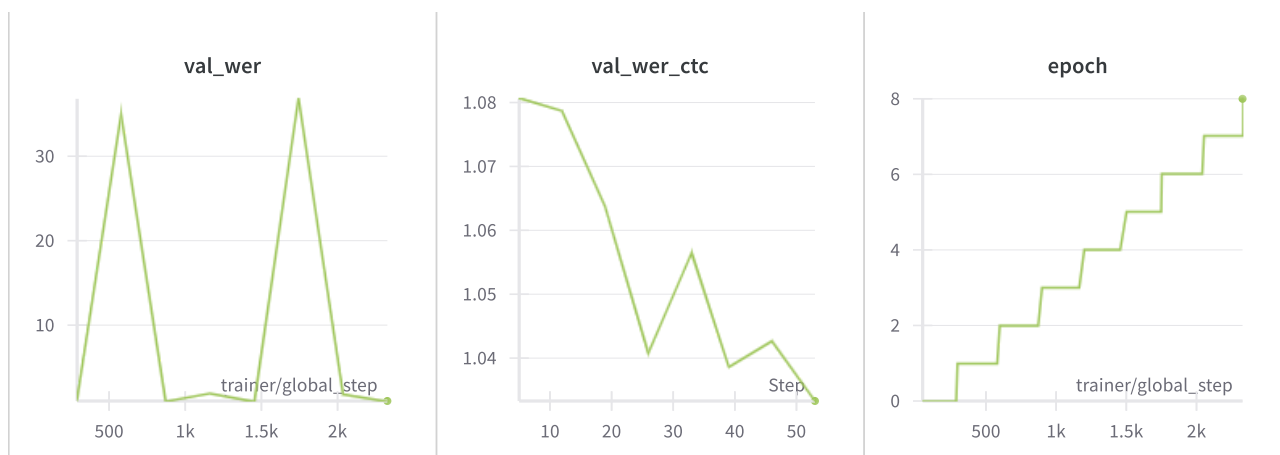
The entire network is trained using a combined loss function that balances the RNNT-loss from the TDT decoder and the CTC-loss from the auxiliary decoder. The hybrid nature of this model enables both sequence-level alignment through RNNT (RNNT Joint module) and frame-level supervision from CTC, aiming to improve ASR performance on limited training data (Rekesh et al., 2023).

▼ Experiment V1 (Baseline Training)

- Optimizer: AdamW
- Learning Rate: 2.5e-3
- Weight Decay: 1e-3
- Batch Size: 128
- Scheduler: NoamAnnealing
- Encoder Frozen: Yes

Training was highly unstable, CTC loss did not stabilize. RNNT loss improved slightly but was still fairly high. **Test WER RNNT = 5.73, Test CTC WER = 1.03.**



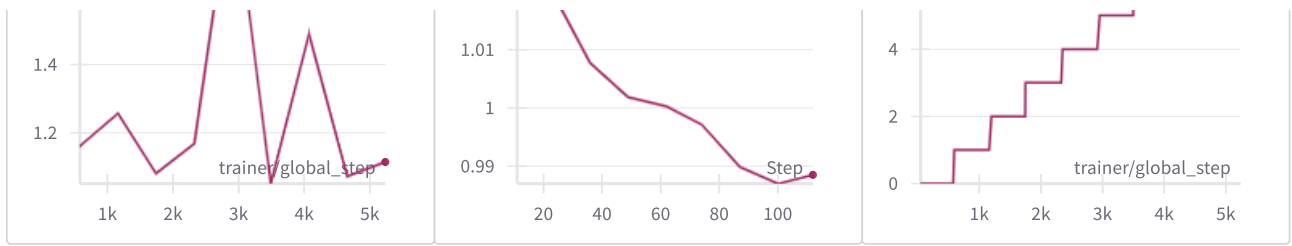


▼ Experiment V2

- Resumed training from V1 with pretty much the same config.
- Encoder Unfrozen, batch size reduced to 64.

The two decoders seem to be on completely different trends. The slow progress suggest that gradients are too small. **Test WER RNNT = 1.00, Test CTC WER = 0.987.**

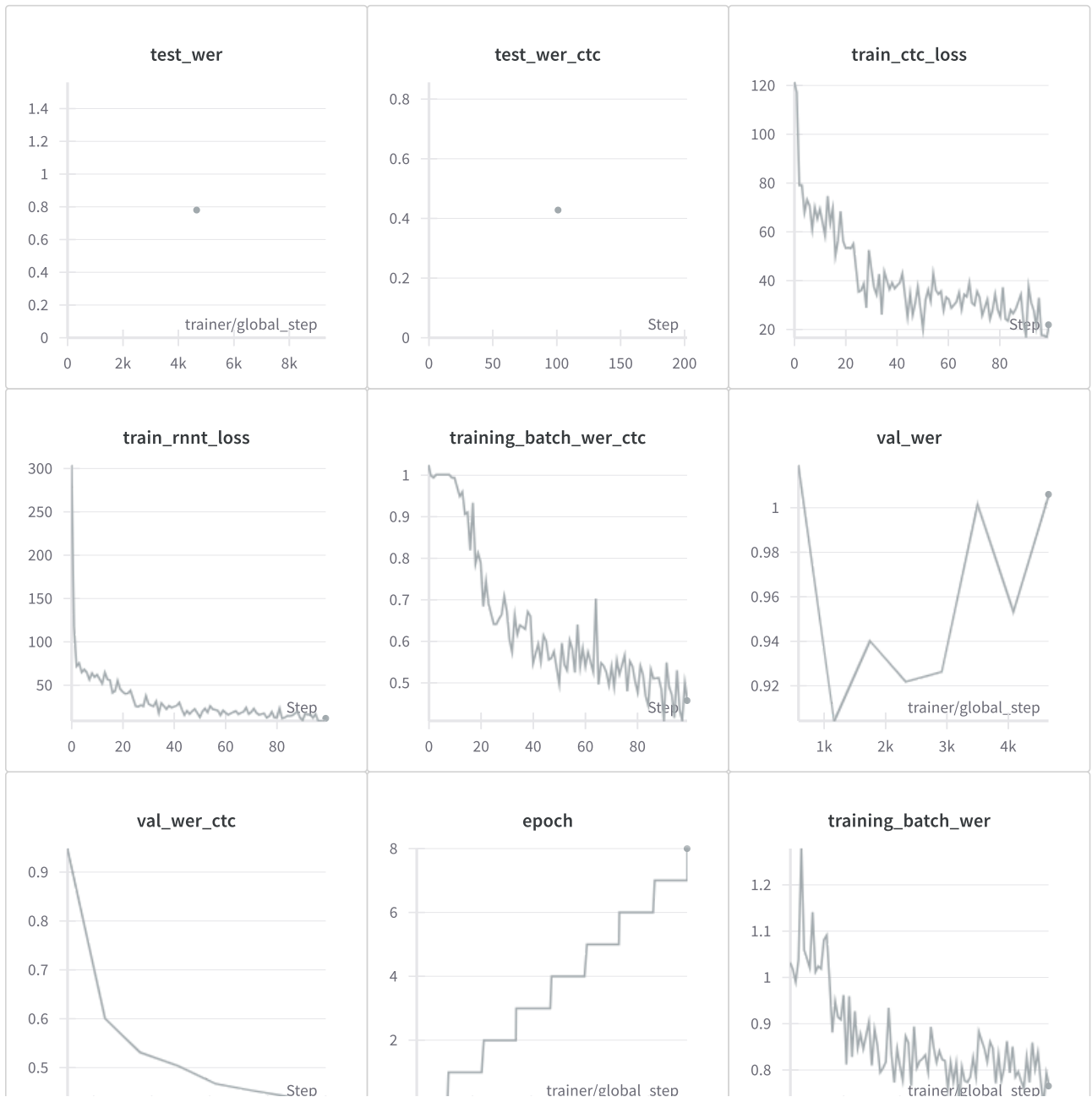




▼ Experiment V3 (New Scheduler & Lower Learning Rate)

- Trained from the original nvidia/parakeet-tdt_ctc-110m
- Switched to WarmupAnnealing.
- Reduced learning rate to $5e-4$, gradient accumulation introduced.

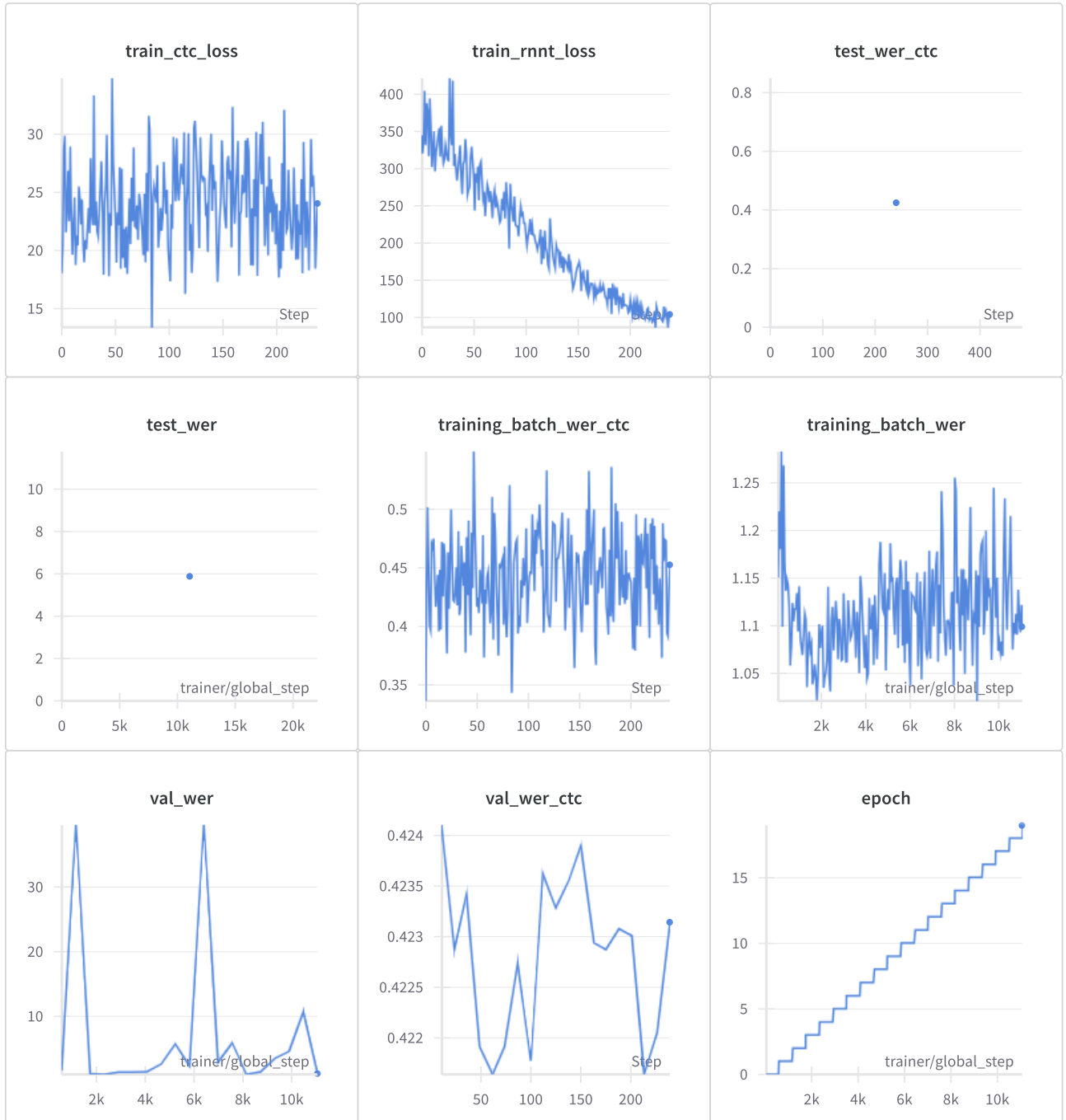
CTC loss showed consistent improvements, but TDT lagged behind, confirming the trend observed in V2. **Test WER RNNT = 0.78, Test CTC WER = 0.42.**





▼ Experiment V4 (Scheduler Evaluation)

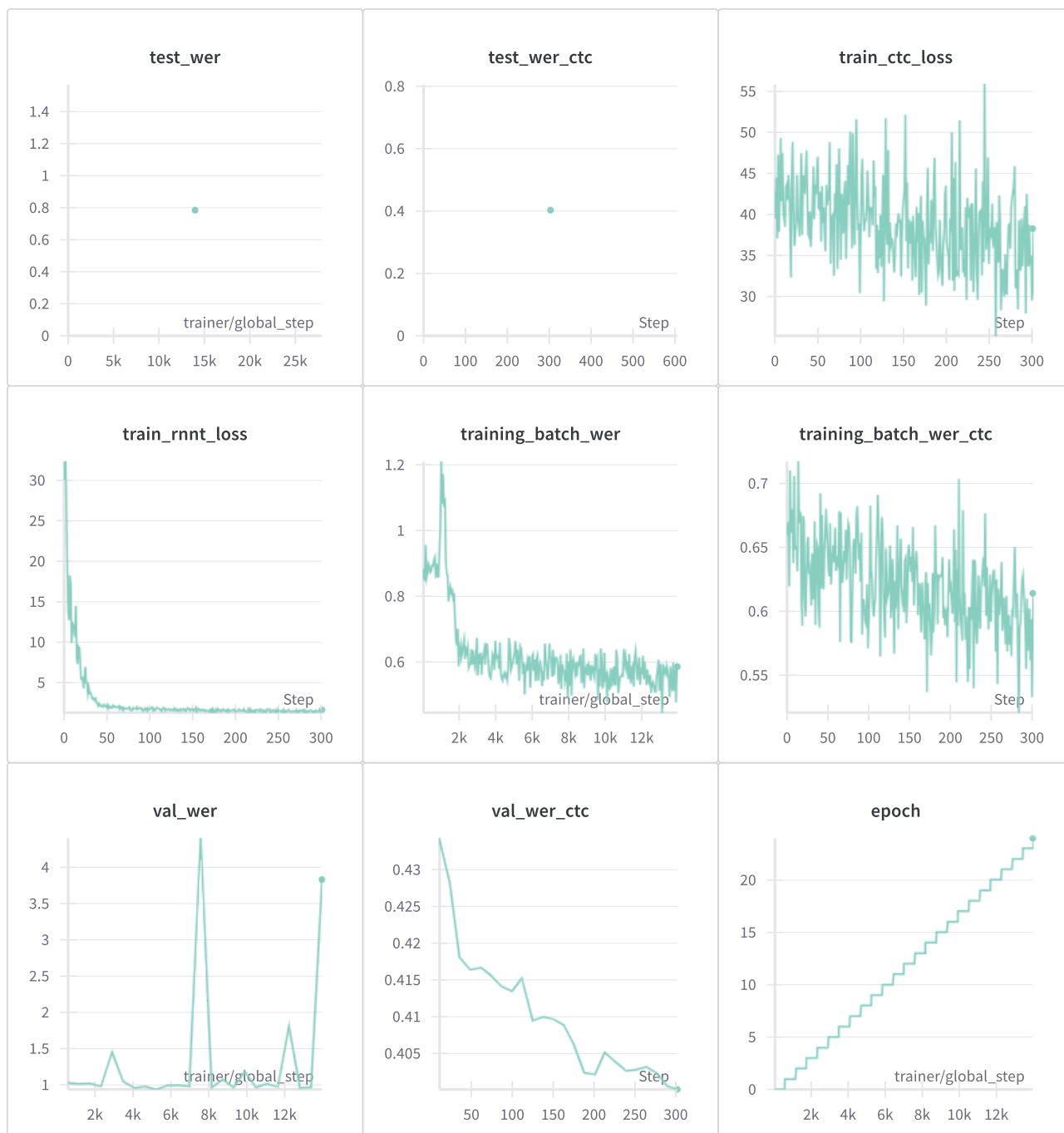
After switching back to NoamAnnealing scheduler, we confirm that it introduced instability in training. The learning curves fluctuated significantly, particularly affecting CTC loss and WER metrics. To address this, we defaulted to WarmupAnnealing, which provided a more stable learning process and led to better convergence in subsequent experiments.



▼ Experiment V5 (Loss Weight Adjustments, resumed from the last checkpoint of experiment 3)

To prioritize the TDT decoder's learning, we decreased the CTC loss weight from 0.3 to 0.15, expecting that this adjustment would provide stronger gradient updates to the whole network and improve the performance of the RNNT decoder. However, this assumption was fundamentally flawed because the RNNT loss was already numerically lower than the CTC loss at the end of experiment 3, meaning that despite having a smaller the CTC loss had the potential to contribute more to the overall combined loss. Decreasing the CTC weight resulted in smaller updates overall since the RNNT loss kept decreasing, reducing the effectiveness of the training process rather than enhancing it.

The discrepancy between validation WER and test WER of the TDT branch is raising for questions about this specific decoder architecture and how it operates during training.



Furthermore, reducing the auxiliary CTC loss weight minimized its stabilizing effect on training. This led to a situation where the RNNT decoder continued to struggle, while the overall network failed to improve as expected. The validation WER for the TDT decoder unexpectedly exploded, while the test WER for the TDT branch remained stagnant at **0.78**, and the test WER for the CTC branch settled at **0.40**. We are still investigating those strange cases of disparity between validation and test WERs.

▼ Experiment V6 (Rebalancing Loss Weights)

To counter the issues observed in V5, we reversed the earlier decision and increased the CTC loss weight instead of decreasing it. This change ensured that the auxiliary CTC decoder's loss function contribute more to be overall loss since it has numerically bigger values even while performing better than the TDT decoder, helping to shake a little bit the whole network with more brutal gradient updates, hence allowing the TDT decoder to learn more effectively.

Additionally, we applied a stronger SpecAugment transformation to prevent the CTC branch from dominating learning and potentially overfit, striking a better balance between generalization and training stability and seeking to harmonize the two decoders' performances. The following SpecAugment configuration was used:

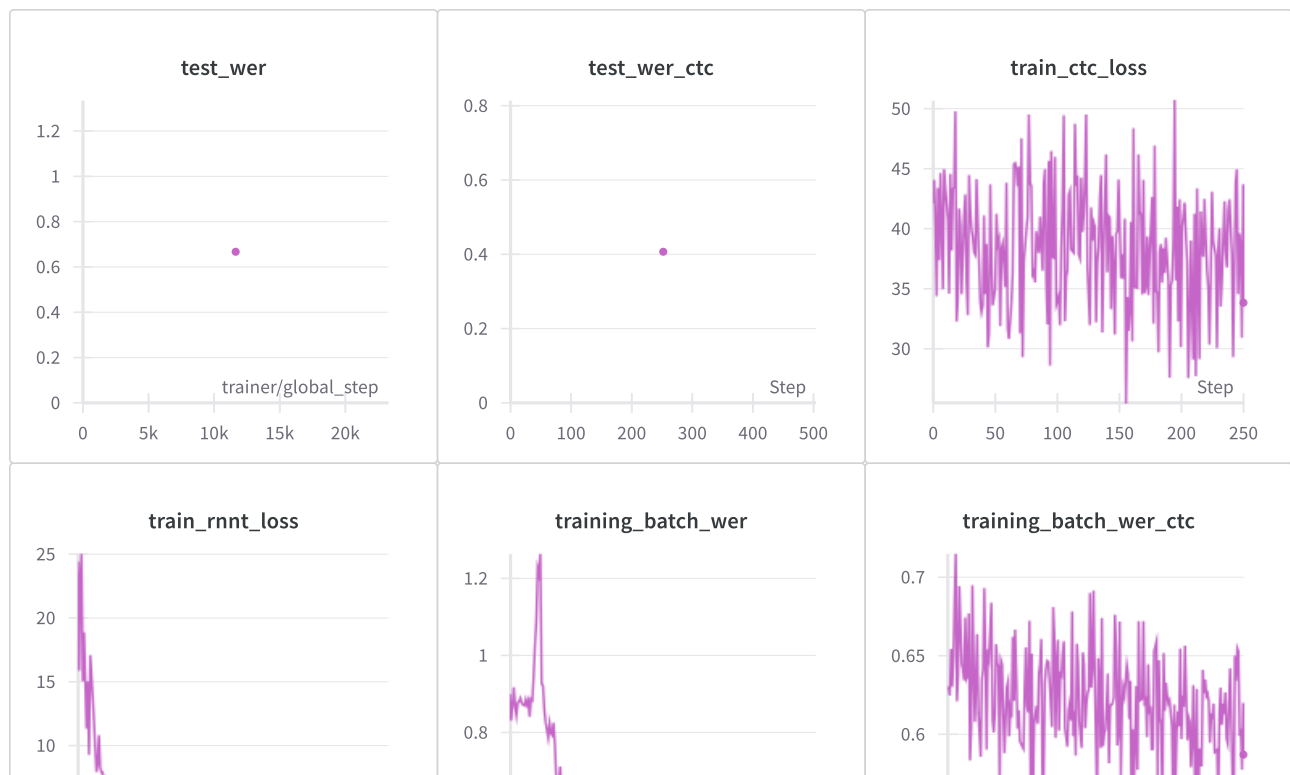
```
model.cfg.spec_augment.freq_masks = 4 # Increase the number of frequency masks 2 -> 4
```

```
model.cfg.spec_augment.freq_width = 27
```

```
model.cfg.spec_augment.time_masks = 10
```

```
model.cfg.spec_augment.time_width = 0.1 # Increase time width 0.05 -> 0.1
```

This setup means that we are applying 4 different frequency masks (4 randomly chosen frequency regions/bands are masked or set to zero) and we increase the time width (Park et al., 2019).





This adjustment significantly reduced fluctuations in training WER for the TDT decoder while preserving the performance of the CTC branch. The model demonstrated improved generalization capacity, and overfitting concerns were mitigated while we now suspect the CTC branch might have started to **underfit**.

As a result, test WER for the RNNT decoder dropped to **0.66**, while test CTC WER expectedly settled at **0.406**. The performance of the TDT decoder in validation keeps raising question though, particularly the validation set was made of ~90% of the same data we used for testing and the performance of the same branch in test are far above its performance in validation mode.

▼ QuartzNet 15x5

▼ Model overview

QuartzNet is a lightweight convolutional neural network (CNN)-based ASR model, designed to be smaller and faster than state-of-the-art models at the time of its publication. QuartzNet employs **1D time-channel separable convolutions**, an efficient implementation of depthwise separable convolutions introduced in the QuartzNet paper (Kriman et al., 2019). It utilizes Connectionist Temporal Classification (CTC) loss for sequence-to-sequence speech recognition and decodes at character level.

The model follows a modular design with five groups of convolutional blocks, each consisting of multiple identical sub-blocks:

- Initial Convolution Layer (C1): Applies a strided convolution with kernel size 33 and 256 output channels.
- Five Main Blocks (B1–B5): Each block consists of 5 convolutional layers with increasing kernel sizes (33, 39, 51, 63, 75) and up to 512 output channels.
- Final Convolutional Layers (C2, C3, C4):
 - C2: Expands feature representation with kernel size 87 and 512 channels.
 - C3: Further processes with kernel size 1 and 1024 channels.

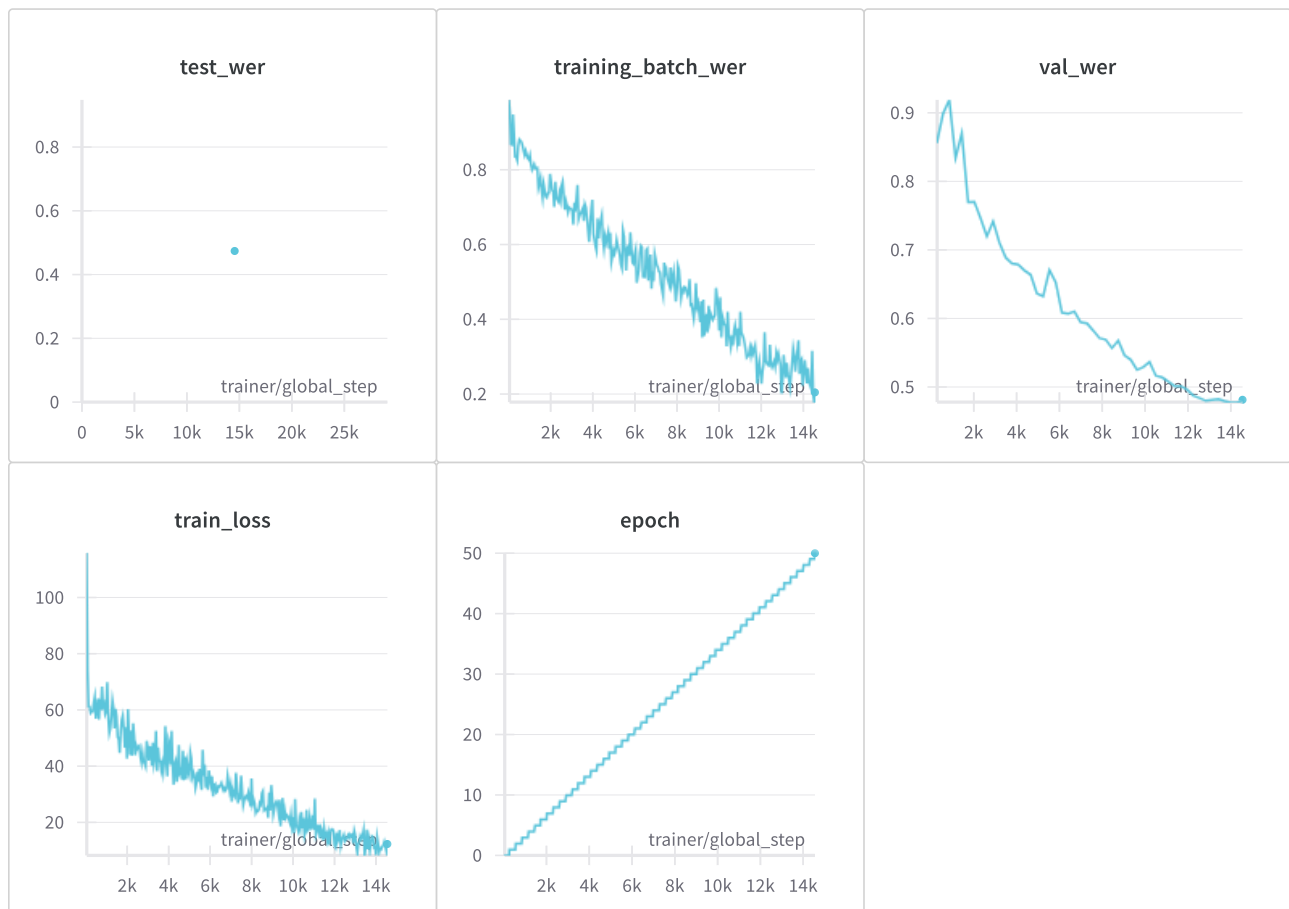
- C4: Maps the output to the number of vocabulary tokens (klabels), ensuring compatibility with CTC decoding.

Each block is repeated multiple times to form different QuartzNet variants for the specific version we fine tuned (QuartzNet-15x5) each group appears three times.

▼ Experiment V1 (Baseline Training)

- Optimizer: Novograd
- Learning Rate: 1e-2
- Batch Size: 128
- Scheduler: CosineAnnealing

Results: Test WER = 0.473 after 50 epochs, but signs of early overfitting.



▼ Experiment V2 (Enhanced SpecAugment for Overfitting Mitigation)

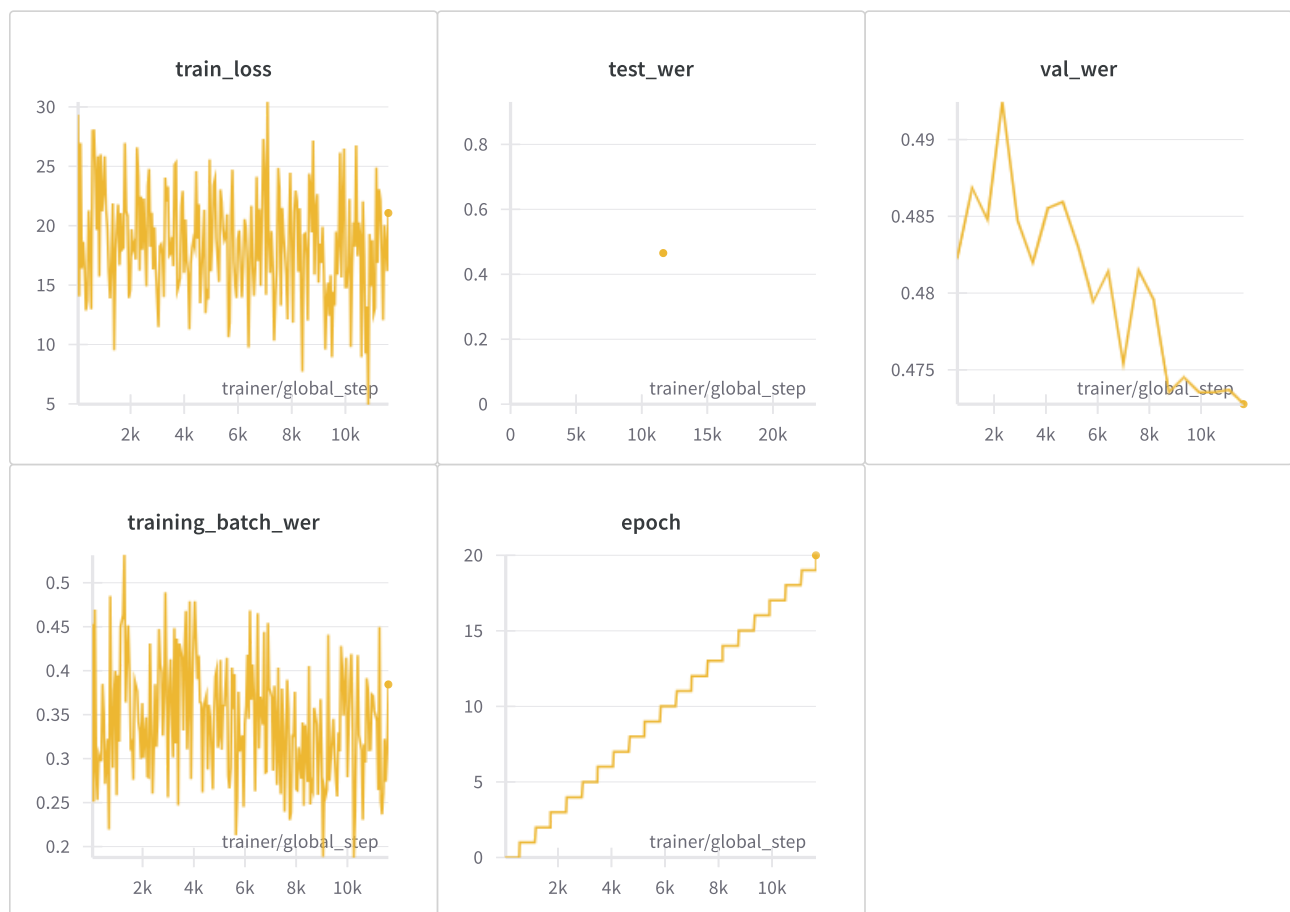
- Optimizer: Novograd
- Learning Rate: 5e-4
- Scheduler: CosineAnnealing
- Max Steps: 14,530
- Warmup Ratio: 0.07
- Min Learning Rate: 1e-6

After observing potential overfitting trends in Experiment V1, this experiment aimed to introduce stronger SpecAugment as a countermeasure while maintaining a reduced learning rate. Training resumed from the V1 model checkpoint. The specAugmentation method used by QuartzNet differs from traditional **time-frequency** masking in a few key ways. Instead of applying independent frequency and time masks, it applies **rectangular masking**, simultaneously masking a contiguous block of the spectrogram in both time and frequency dimensions. For this experiment, we just increased the number of rectangular masks to be applied (compared to QuartzNet15x5's default SpecAugment Config):

```
model.cfg.spec_augment.rect_freq = 50
```

```
model.cfg.spec_augment.rect_time = 120
```

```
model.cfg.spec_augment.rect_masks = 10 # Increased
```



The impact of enhanced SpecAugment was noticeable: training batch WER stabilized slightly above previous values (~0.38 instead of ~0.25), while test WER improved by an average of 0.06 points. This validated the hypothesis that the model was indeed overfitting in V1, as increased regularization improved generalization without negatively impacting training WER.

After 20 training epochs, the final model achieved a test **WER of 0.465**, demonstrating better robustness compared to earlier experiments.

▼ Parakeet TDT-1.1B (XXL Model)

▼ Model Overview

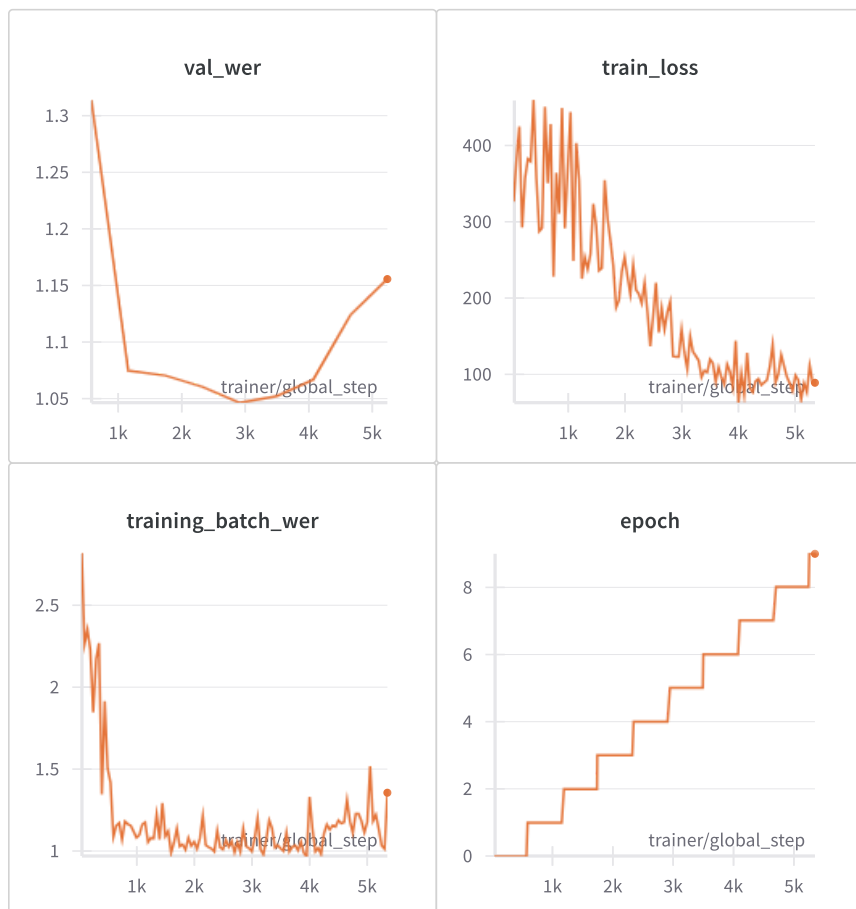
Parakeet TDT-1.1B is a large-scale ASR model built upon Fast Conformer, which is an optimized version of the original Conformer model. It includes 8x depthwise-separable convolutional downsampling and a more efficient subsampling module (Rekesh et al., 2023).

This model utilizes Fast Conformer encoder as encoder and has a TDT decoder, which operates as a Recurrent Neural Network Transducer (RNNT) with Token-and-Duration alignment, jointly predicting token and duration (Xu et al, 2023).

Despite its large size, the model struggled to converge, while larger models typically converge faster this scenario was to be expected after the feedbacks we got from the TDT branch of the hybrid model. It's very likely that something in the Token-and-Duration Transducer architecture is particularly sensitive to some of the issues of our training dataset (probably the misalignment problem that affects a few examples in the dataset).

▼ Experiment V1 (Baseline Training)

- Optimizer: AdamW
- Learning Rate: 5e-4
- Batch Size: 16 (Gradient Accumulation = 4)
- Scheduler: NoamAnnealing



▼ Experiment V2 (More SpecAugment & Alternative Scheduler)

- Switched to WarmupAnnealing.
- learning rate: 5e-5
- Batch size increased to 50.
- warmup_ratio: 0.1
- min_lr: 1e-9

The following SpecAugment Configuration was used:

```
# Increase SpecAugment for larger models to prevent overfitting
```

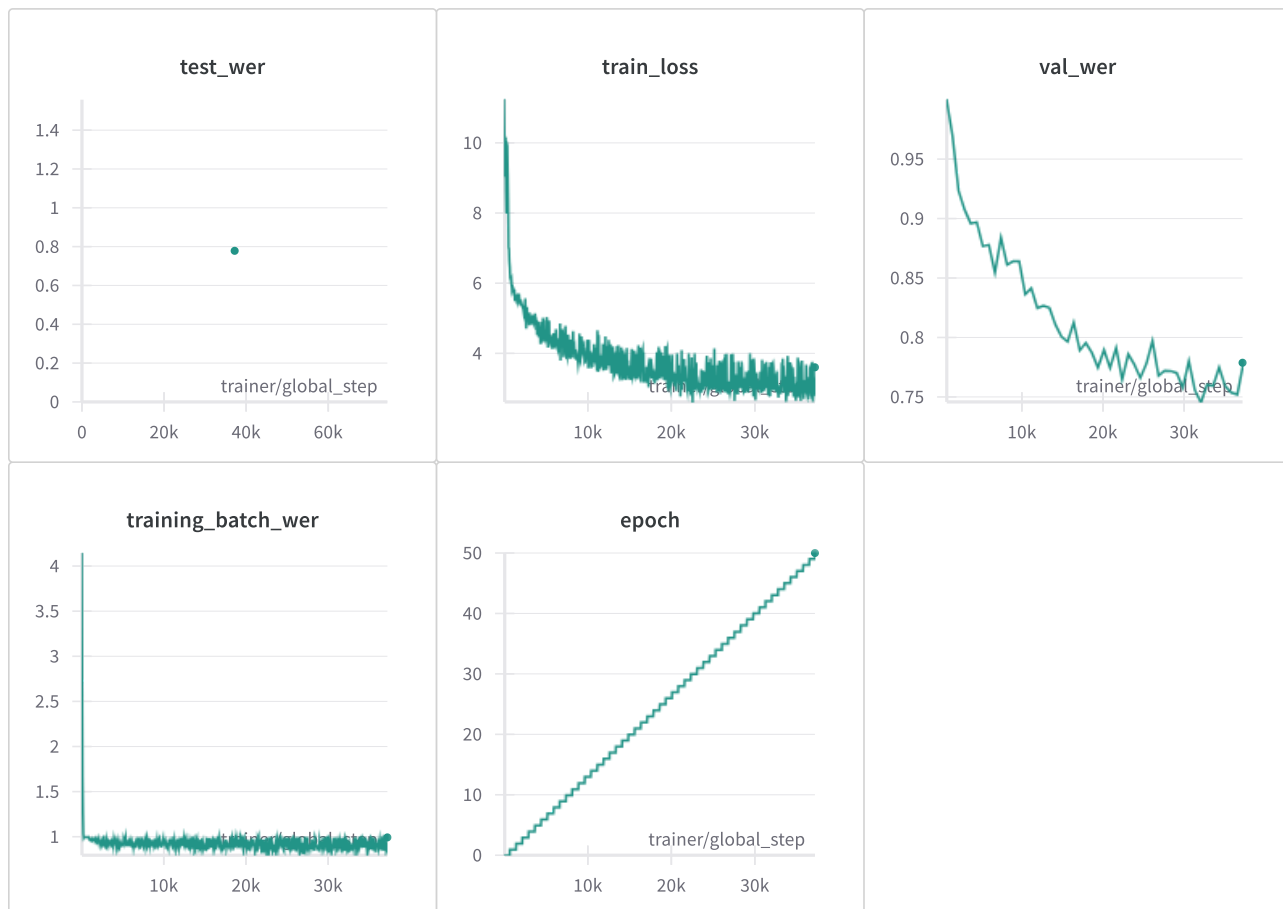
```
model.cfg.spec_augment.freq_masks = 4 # Increase the number of frequency masks
```

```
model.cfg.spec_augment.freq_width = 27
```

```
model.cfg.spec_augment.time_masks = 15 # Increase the number of time masks
```

```
model.cfg.spec_augment.time_width = 0.1 # Increase time width
```

Compared to experiment V6 of training the hybrid tdt-ctc model we just increased the number of randomly selected time regions to be masked (set to 0).



After 50 epochs, **Test WER = 0.77**, but training batch WER remained high (~0.90), indicating underfitting.

We suspect that the TDT decoders might rely heavily on the alignment quality in the training data due to the fact that it tries to predict tokens and align them with predicted durations. That could explain the struggle we observed with this decoder architecture and why it seems to perform better on test than training.

▼ Conclusion

The models trained in [Experiment V6](#) (Hybrid Parakeet-114M TDT-CTC) and [Experiment V2](#) (QuartzNet-15x5) have been published on Hugging Face, making them accessible for further research and practical deployment. The Hybrid Parakeet model was trained for a total of **27 epochs (cumulated number of epochs of V3 and V6)**, achieving a final Test WER of **0.66 (TDT)** and **0.406 (CTC)**, while the QuartzNet model underwent **70 epochs of training**, with a final Test WER of **0.465**. All training configurations, hyperparameters, and scripts used for these experiments are publicly available in the following repository:

 [Bambara-ASR Repository](#).

▼ Key Findings

- **Smaller models are enough:** if we are not trying to make a multilingual model then a model of 20M to 100M parameter is just what we need given the limited amount of data.
- **Learning Rate Scheduler choice significantly impacts stability and performance in low resource settings.**
- **Simple is better than complex:** although the hybrid Parakeet-tdt-ctc-114m ultimately outperformed the QuartzNet model with its CTC decoder, the latter is much more interpretable and provides more insights for future trainings.

▼ References

1. Krizan, S., Beliaev, S., Ginsburg, B., Huang, J., Kuchaiev, O., Lavrukhin, V., Leary, R., Li, J., & Zhang, Y. (2019). QuartzNet: Deep Automatic Speech Recognition with 1D Time-Channel Separable Convolutions. arXiv preprint arXiv:1910.10261. <https://arxiv.org/abs/1910.10261>
2. Rekesh, D., Koluguri, N. R., Krizan, S., Majumdar, S., Noroozi, V., Huang, H., Hrinchuk, O., Puvvada, K., Kumar, A., Balam, J., & Ginsburg, B. (2023). Fast Conformer with Linearly Scalable Attention for Efficient Speech Recognition. arXiv preprint arXiv:2305.05084. <https://arxiv.org/abs/2305.05084>
3. Xu, H., Jia, F., Majumdar, S., Huang, H., Watanabe, S., & Ginsburg, B. (2023). Efficient Sequence Transduction by Jointly Predicting Tokens and Durations. arXiv preprint arXiv:2304.06795. <https://arxiv.org/abs/2304.06795>
4. Diarra, S., Leventhal, M., & Tapo, A. A. (2022). RobotsMali Griots Speech Dataset, and ASR. Retrieved from <https://github.com/robotmali-ai/jeli-asr/>

5. Park, D. S., Chan, W., Zhang, Y., Chiu, C.-C., Zoph, B., Cubuk, E. D., & Le, Q. V. (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. In Interspeech 2019. ISCA.
[DOI: 10.21437/Interspeech.2019-2680](http://dx.doi.org/10.21437/Interspeech.2019-2680)

Created with  on Weights & Biases.

<https://wandb.ai/yacoudiarra-wl/bam-asr-nemo-training/reports/Draft-Technical-Report-V1--VmlldzoxMTlyOTMzOA>